

IOT AND M2M

M2M

Machine to Machine (M2M) refers to networking of machines for the purpose of remote monitoring, control, and data exchange. The end-to-end architecture for M2M systems comprises M2M area networks, a communication network, and an application domain.

An M2M area network comprises machines that have embedded hardware modules for sensing, actuation, and communication. Various communication protocols can be used for M2M local area networks such as Zigbee, Bluetooth, Modbus, M - Bus, Wireless, Power Line Communication, and 6LoWPAN.

These communication protocols provide connectivity between M2M nodes within an M2M area network. The communication network provides connectivity to remote M2M area networks. The communication network can use wired or wireless networks. The M2M area network uses either proprietary or non - IP based protocols.

Communication between M2M nodes and the M2M Gateway is based on the communication protocol used. The M2M Gateway performs protocol translation to enable IP connectivity for M2M. The M2M Gateway acts as a proxy, performing translation from / to native protocols to the M2M area network.

M2M data is gathered into point solutions such as enterprise applications and service management applications for remote monitoring. M2M has various application domains such as smart metering, home automation, industrial automation, and smart grid.

DIFFERENCE BETWEEN IOT AND M2M

The differences between M2M and IoT are described as follows:

Communication Protocols:

M2M and IoT differ in how communication between machines or devices happens. M2M uses proprietary or non - IP based communication protocols within M2M area networks. Commonly used M2M protocols include Zigbee, Bluetooth, Modbus, Wireless M - Bus, and Power Line Communication.

The focus of communication in M2M is usually on protocols below the network layer. The focus of communication in IoT is usually on protocols at the network layer such as HTTP, Web Sockets, MQTT, XMPP, DDS, and AMQP.

Machines in M2M vs Things in IoT:

The "things" in IoT refer to physical objects that have unique identifiers and can sense and communicate with the external environment or their internal physical status. The unique identifiers of things in IoT are IP addresses.

Things have software components for accessing, processing, and storing sensor information and for controlling actuators. IoT systems can include IoT devices of various types such as fire alarms, door alarms, and lighting control devices.

Hardware versus Software Emphasis:

While the emphasis of M2M is more on hardware with embedded modules, the emphasis of IoT is more on software. IoT devices run specialized software for sensor data collection, data analysis, and interfacing with the cloud through IP - based communication.

Data Collection and Analysis:

M2M data is collected in point solutions and often stored in on - premises infrastructure. In contrast, data in IoT is collected in the cloud.

The analytical component analyzes the data and stores the results in the cloud database. Data and analysis results are visualized using cloud - based applications. The centralized controller is aware of the status of all nodes and sends control commands to them.

Applications:

M2M data is collected in point solutions and can be accessed by on - premises applications such as diagnostic applications, service management applications, and enterprise applications.

SDN AND NFV FOR IOT

Software Defined Networking (SDN) and Network Function Virtualization (NFV) are important technologies and have applications in IoT.

SOFTWARE DEFINED NETWORKING

Software Defined Networking (SDN) is a networking architecture that separates the control plane from the data plane and centralizes the network controller. Conventional network architecture is built with specialized hardware such as switches and routers.

Network devices in conventional architectures are becoming increasingly complex due to the growing number of distributed products and the use of proprietary hardware and interfaces.

The control plane is the part of the network that carries signaling and routing message traffic, while the data plane is the part of the network that carries payload data traffic.

Limitations of Conventional Network Architecture:

Complex Network Devices:

Interoperability is limited due to the lack of standard and open interfaces. Network devices use proprietary hardware and software and have slow product lifecycles, limiting innovation.

Conventional networks were well suited for static traffic patterns and had many products designed for specific applications. In cloud computing environments, traffic patterns are more dynamic. Due to the complexity of conventional network devices, making changes to meet dynamic traffic patterns has become increasingly difficult.

Management Overhead:

Conventional networks involve significant management overhead. Network managers find it increasingly difficult to manage multiple network devices and interfaces from multiple vendors. Upgrades and network configuration changes are required on multiple devices.

Limited Scalability:

Virtualization technologies used in cloud computing environments have increased the number of hosts requiring network access. IoT applications hosted in the cloud are distributed across multiple virtual machines that require traffic exchange.

Components of IoT applications run distributed algorithms on a large number of virtual machines that require huge amounts of data exchange.

Such computing environments require highly scalable and easy - to - manage network architectures with minimal manual configuration, which is increasingly difficult with conventional networks.

KEY ELEMENTS OF SDN

Centralized Network Controller:

With decoupled control and data planes and a centralized network controller, the network administrator can rapidly configure the network. SDN applications can be deployed using programmable open APIs. This speeds up innovation, as network features no longer depend on vendors embedding them in proprietary hardware.

Programmable Open APIs:

SDN architecture proposes programmable open APIs for the interface between the SDN application and control layers. With these open APIs, various network services can be implemented such as routing, quality of service, and access control.

Standard Communication Interface (OpenFlow):

SDN architecture uses a standard communication interface between the control and infrastructure layers. OpenFlow, which is defined by the Open Networking Foundation, is the widely accepted SDN protocol for the southbound interface.

OpenFlow allows the forwarding plane of network devices to be directly accessed and manipulated.

OpenFlow uses the concept of flow entries to direct network traffic based on defined rules. Flows can be programmed statically or dynamically by the SDN controller software.

An OpenFlow switch comprises one or more flow tables and group tables that perform packet lookup and forwarding, and an OpenFlow channel to an external controller. The OpenFlow protocol is implemented on both sides of the interface between the controller and the network devices.

NETWORK FUNCTION VIRTUALIZATION

Network Function Virtualization (NFV) is a technology that leverages virtualization to consolidate heterogeneous network devices onto industry - standard high - volume servers, switches, and storage.

NFV is complementary to SDN, as NFV can provide the infrastructure on which SDN can run. NFV and SDN are mutually beneficial but not dependent on each other. Network functions can be virtualized without SDN.

ELEMENTS OF ARCHITECTURE

Virtualized Network Function (VNF):

A VNF is a software implementation of a network function that is capable of running over the NFV infrastructure.

NFV Infrastructure (NFVI):

NFVI includes compute, network, and storage resources that are virtualized.

NFV Management and Orchestration:

Orchestration focuses on virtualization - specific management tasks and covers the orchestration and lifecycle management of physical and / or software resources that support infrastructure utilization and the lifecycle management of VNFs.

NFV comprises network functions implemented in software that run on virtualized resources in the cloud. NFV enables separation of network functions implemented in software from the underlying hardware.